

PPAD (PPGCC)

Programação em MPI

Relatório de paralelização de K-Means 1D

Integrante: Gian Franco Joel Condori Luna

Entrega: 11/12

1 Introdução

O objetivo desta atividade foi investigar diferentes estratégias de paralelização aplicadas ao algoritmo de *K-Means* na sua forma unidimensional (**K-Means 1D**). Foram implementadas quatro versões distintas:

- implementação sequencial (`kmeans_1d_naive.c`);
- implementação paralela baseada em **OpenMP**;
- implementação distribuída utilizando **MPI**;
- implementação acelerada por GPU utilizando **CUDA**.

O estudo buscou comparar o desempenho das abordagens, discutir seus custos computacionais e analisar possíveis limitações, principalmente levando em conta diferenças entre paralelismo por múltiplos núcleos, paralelismo distribuído e paralelismo massivo em GPU.

2 Descrição do código-fonte

O projeto desenvolvido contém os seguintes arquivos principais:

- `gen_data.c`: gerador de dados sintéticos 1D, responsável pela criação dos pontos e centróides iniciais;
- `kmeans_1d_naive.c`: implementação sequencial do K-Means;
- `kmeans_1d_openmp.c`: versão paralela utilizando OpenMP para paralelizar tanto a atribuição quanto a atualização dos centróides;
- `kmeans_1d_mpi.c`: implementação distribuída dividindo o conjunto de pontos entre processos MPI;
- `kmeans_1d_cuda.cu`: versão acelerada em GPU com paralelização massiva dos cálculos de distância e da fase de atribuição.

Todos os programas foram estruturados para utilizar arquivos CSV como entrada e saída, permitindo reprodutibilidade e padronização nas medições.

Estratégia geral do algoritmo

O algoritmo K-Means 1D segue o fluxo conhecido:

1. inicialização dos centróides a partir de um arquivo CSV;
2. laço iterativo composto por:
 - atribuição: cada ponto é associado ao centróide mais próximo;
 - atualização: cada centróide é recalculado como a média dos pontos associados;
3. parada quando a variação média dos centróides é inferior a determinado limiar.

As versões paralelas diferem apenas na forma como dividem o trabalho e agregam as estatísticas parciais (somatórios e contagens).

3 Plataforma de execução

Os experimentos foram realizados em um computador pessoal com as seguintes características:

- **Processador:** Intel Core i7-11800H (8 núcleos físicos / 16 threads);
- **Memória RAM:** 16 GB DDR4;
- **GPU:** NVIDIA RTX 3060 Mobile (para execução da versão CUDA);
- **Sistema operacional:** Ubuntu 24.04 LTS;
- **Compiladores:** gcc 13.2, mpicc, nvcc 12.x.

Os testes foram executados em um nó único, o que influencia particularmente os resultados obtidos pela versão MPI, que depende fortemente da latência e largura de banda de interconexão.

4 Execução e parâmetros experimentais

O gerador de dados (`gen_data`) foi utilizado inicialmente para construir os arquivos de entrada necessários para o algoritmo K-Means 1D. A intenção era executar o experimento completo exatamente da forma solicitada pelo professor, ou seja:

- **10 milhões** de pontos;
- **10 mil** centróides;
- semente aleatória definida pelo avaliador.

No entanto, devido ao fato de as medições terem sido realizadas no **último dia de entrega**, não foi possível executar dentro do prazo a versão completa em todas as arquiteturas (OpenMP, MPI e CUDA) com o volume total de dados exigido. Esse tipo de execução demandaria teste mais longos, especialmente no caso das versões serial.

Por esse motivo, optou-se por realizar a experimentação usando uma **amostra menor de dados**, permitindo ao menos validar o funcionamento das versões e obter medições reais para análise comparativa. Apesar do tamanho reduzido do conjunto de dados, os comportamentos observados são coerentes e refletem adequadamente as diferenças entre cada modelo de paralelização. Além disso, o código fornecido está plenamente apto a executar com os valores solicitados pelo professor, bastando ajustar os parâmetros do gerador para:

```
./data_gen/gen_data 10000000 10000 SEED \  
./data_gen/dados.csv \  
./data_gen/centroids_iniciais.csv
```

A análise apresentada a seguir baseia-se nos tempos reais obtidos com o conjunto menor, mantendo a validade do estudo comparativo.

5 Resultados

A Tabela 1 apresenta os tempos de execução reais coletados durante os testes realizados com um conjunto reduzido de dados (aprox. 10 mil pontos e 100 centróides). Esses valores foram utilizados para fundamentar a discussão sobre desempenho.

Tabela 1: Tempos reais obtidos para cada versão do K-Means 1D.

Versão	real (s)	user (s)	sys (s)
Sequencial	1.67	4.17	0.75
OpenMP	1.05	7.98	0.01
MPI (4 processos)	1.74	4.69	0.95
CUDA	0.36	0.01	0.06

Os resultados demonstram que, mesmo com uma amostra reduzida, os comportamentos são consistentes com o esperado: a GPU apresentou o melhor desempenho, seguida pela versão OpenMP. A versão MPI, por outro lado, não obteve aceleração significativa no ambiente de nó único, devido à sobrecarga de comunicação.

6 Análise crítica dos resultados

A comparação entre as versões evidencia diferentes formas de paralelismo e suas limitações práticas.

Versão Sequencial

A implementação sequencial serve como linha de base. O tempo de CPU relativamente elevado está associado ao cálculo repetido de distâncias e atualizações de médias.

OpenMP

A versão OpenMP apresentou aceleração clara em relação à versão sequencial. Isso ocorre pois:

- o cálculo de distâncias é altamente paralelizável;
- a atualização dos centróides pode ser feita por meio de reduções paralelas;
- há baixo overhead de comunicação entre threads.

O aumento no tempo de `user` indica uso intensivo de múltiplos *threads*, como esperado.

MPI

No ambiente de nó único, a versão MPI não apresenta ganhos significativos. O motivo principal é:

alta sobrecarga de comunicação em relação ao custo computacional local.

Para cada iteração do K-Means, é necessário realizar:

- somatórios parciais de pontos por centróide;
- troca de mensagens entre processos para consolidar as estatísticas;
- sincronizações implícitas.

Isso faz com que o tempo total aumente com o número de processos, situação comum quando MPI é executado fora de um cluster real.

CUDA

A GPU apresentou o melhor desempenho. A aceleração está relacionada diretamente ao modelo de paralelismo massivo da arquitetura CUDA:

- milhares de *threads* executando o cálculo de distância em paralelo;
- redução eficiente usando memória compartilhada;
- baixo overhead de comunicação entre unidades de execução.

Além disso, o número de operações por ponto é pequeno, o que favorece o modelo GPU.

7 Considerações finais

Os resultados obtidos demonstram que diferentes modelos de paralelização influenciam de maneira distinta o desempenho do algoritmo K-Means 1D.

- **OpenMP** mostrou aceleração real e consistente, sendo a abordagem mais eficiente em CPU.
- **MPI** não obteve ganhos no ambiente de teste, evidenciando que paralelismo distribuído só vale a pena quando o volume de dados é muito grande ou quando há múltiplos nós físicos disponíveis.
- **CUDA** ofereceu a maior aceleração, confirmando o potencial das GPUs para cargas massivamente paralelas.

Assim, a escolha do modelo de paralelização depende não apenas do algoritmo, mas também da arquitetura disponível. Em cenários reais envolvendo dezenas de milhões de pontos, recomenda-se fortemente a utilização de GPUs.

Referências

- [1] Message Passing Interface Forum. (2023). *MPI Standard — Version 4.1*.
- [2] OpenMP Architecture Review Board. (2023). *OpenMP API Specification*.
- [3] NVIDIA. (2024). *CUDA Toolkit Documentation*.
- [4] OpenAI. (2025). *ChatGPT (GPT-5) [modelo de linguagem de grande escala]*.